

MATLAB ASSIGNMENT

BEM GROUP ONE

Name	Registration No.
Bunoga Mustafa	BU/UP/2025/5149
Masiga Joel	BU/UP/2025/4644
Lubega Mark	BU/UP/2025/4641
Agulu Mevian Evenah	BU/UP/2025/5149
Wanyira Samuel	BU/UP/2025/4662

Newton Raphson Methods

find the root of the equation

$$f(x) = 1 - 3x + \frac{1}{2}xe^x$$

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

$$x_{n+1} = x_n - \left(1 - 3x_n + \frac{1}{2}x_ne^{x_n}\right) / \left(\frac{1}{2}(1 + x_n)e^{x_n} - 2\right)$$

This problem requires determining the zero of the given function to 5 dps. Hence the tolerance, of the problem is error ≤ 0.000005

when

$$x_0 = 0.5$$

n	x_n	$ x_{n+1} - x_n $
0	0.500000	
1	0.450200	0.0498
2	0.451541	0.001341
3	0.451542	0.000001
4	0.451542	0.000000

Hence the zero of the function close to 0.5 is 0.45154 (to 5 dps)

For

$$x_0 = 1.6$$

n	x_n	$ x_{n+1} - x_n $
0	1.600000	
1	1.552769	0.047231
2	1.549552	0.003231
3	1.549538	0.000014
4	1.549538	0.000000

Hence the zero of the function close to 1.60 is 1.54954 (to 5 decimal places)

MATLAB code for solving the equation using newton Raphson using a for loop.

```
clc;

clear;

% Define the function and its derivative
f=1-3*x +0.5*x*expx;
df=0.5*(1 + x) *expx -3;

%initial guesses for the two roots
initial_guesses = [0.5,1.6];

tol = 0.0001;
maxlter = 10;
for j=1:2
    x=initial_guesses (j);
    fprintf ('\nFinding root %d\n', j);
end

for i = 1:maxlter
    % Newton-Raphson formula
    x_new = x-f(x)/df(x);
    %Calculate error
    error = abs(x_new - x);
    fprintf('Iteration %d:x = %.6f, Error = %.6\n',i,x_new, error);
    x = x_new;
    %Check convergence
    if error < tol
        break;
    end
end
```

```
fprintf('Root %d = %.6f\n',j,x);
```

MATLAB code for solving the equation using newton Raphson using a while loop

```
clc;
```

```
clear;
```

```
% Define the function and derivative
```

```
f = @(x) x^2 - 9;
```

```
df = @(x) 2*x;
```

```
% Initial guesses for the two roots
```

```
initial_guesses = [2, -2];
```

```
tol = 0.0001;
```

```
maxIter = 10;
```

```
j = 1;
```

```
while j <= 2
```

```
    x = initial_guesses(j);
```

```
    error = inf;
```

```
    i = 0;
```

```
    fprintf('\nFinding root %d\n', j);
```

```
    while error >= tol && i < maxIter
```

```
        % Newton-Raphson formula
```

```
        x_new = x - f(x)/df(x);
```

```
        % Calculate error
```

```
        error = abs(x_new - x);
```

```
        i = i + 1;
```

```
        fprintf('Iteration %d: x = %.6f, Error = %.6f\n', ...  
            i, x_new, error);
```

```
        x = x_new;
```

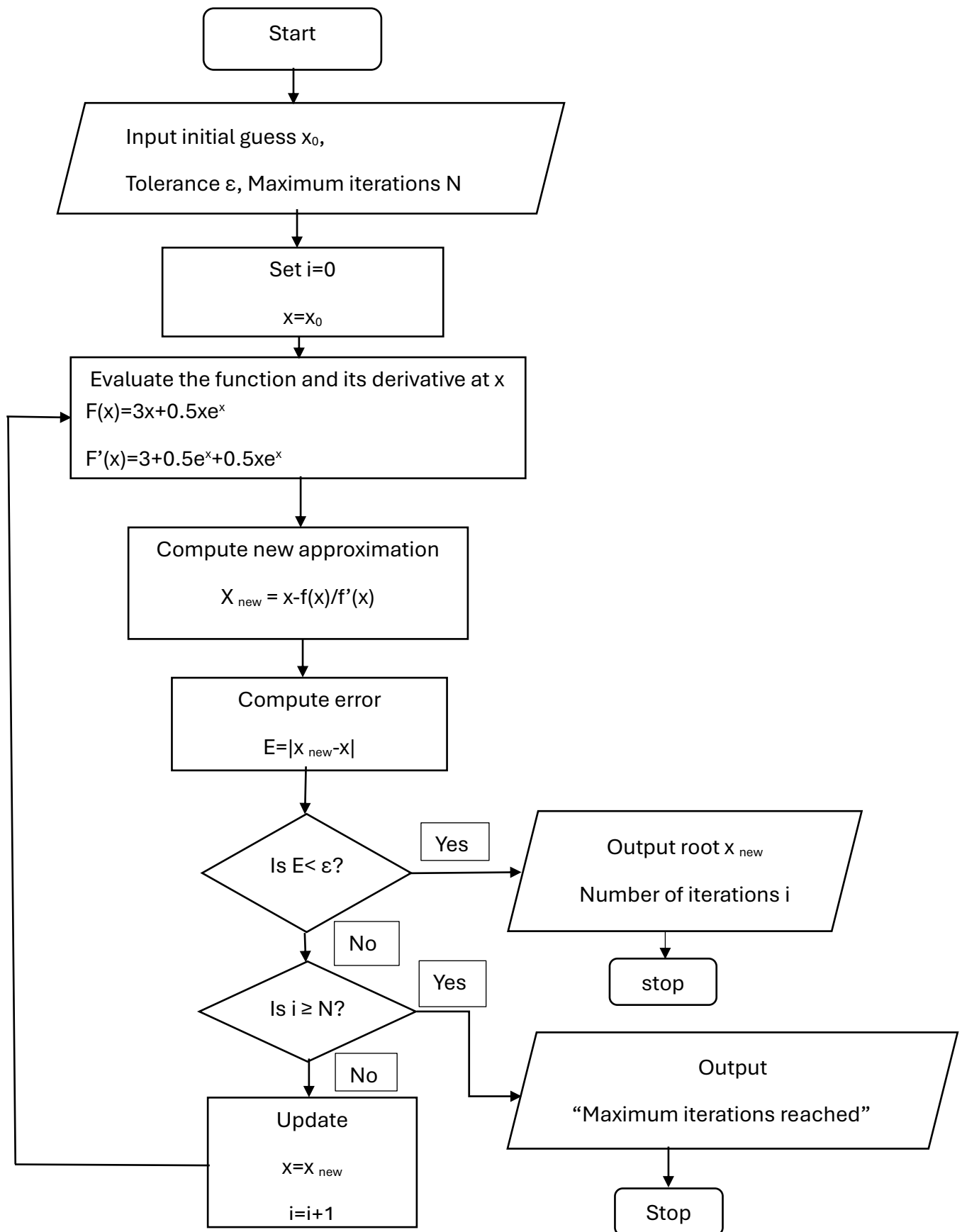
```
    end
```

```
    fprintf('Root %d = %.6f\n', j, x);
```

```
    j = j + 1;
```

```
end
```

FLOWCHART FOR THE NEWTON RAPHSON METHOD



Pseudo code for the newton Raphson chart

START

Define $f(x) = 3x + 0.5x \cdot \exp(x)$

Define $df(x) = 3 + 0.5 \cdot \exp(x) \cdot (1+x)$

Input x_0 , tolerance TOL, maximum iterations N

Set $i = 1$

WHILE $i \leq N$

 Calculate $x_1 = x_0 - f(x_0)/df(x_0)$

 IF $\text{abs}(x_1 - x_0) < \text{TOL}$ THEN

 Display x_1

 STOP

 END IF

 Set $x_0 = x_1$

 Set $i = i + 1$

END WHILE

Display "Maximum iterations reached"

STOP

THE SECANT METHOD

Find the positive solution of

$$f(x) = x - 2\sin x = 0$$

by the secant method, starting with

$$x_{n+1} = x_n - \frac{(x_n - 2\sin x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})} = x_n - \frac{N_n}{D_n}$$

n	x_{n-1}	x_n	N_n	D_n	$ x_{n+1} - x_n $
1	2.000000	1.900000	-0.000740	-0.174005	0.004253
2	1.900000	1.895747	-0.000002	-0.0006986	0.000252
3	1.895747	1.895494	0	0	0

Hence the root is 1.895494 (correct to 6dps)

MATLAB code for solving the equation using secant method.

Secant Method Using a FOR Loop

```
clc;
clear;

% Define the function
f = @(x) x - 2*sin(x);

% Initial guesses
x0 = 1;
x1 = 2;

% Tolerance and maximum iterations
tol = 1e-6;
maxIter = 100;

% Secant method using FOR loop
for i = 1:maxIter

    % Evaluate the function
    f0 = f(x0);
    f1 = f(x1);
```

```

% Check for division by zero
if f1 - f0 == 0
    disp('Division by zero. Method failed.');
```

$$x_2 = x_1 - f_1 \cdot (x_1 - x_0) / (f_1 - f_0);$$

```

    break;
end

% Calculate the next approximation

% Check convergence
if abs(x2 - x1) < tol
    fprintf('Root = %.8f\\n', x2);
    fprintf('Iterations = %d\\n', i);
    break;
end

% Update the guesses
x0 = x1;
x1 = x2;
end

% Check whether maximum iterations were reached
if i == maxIter && abs(x2 - x1) >= tol
    disp('Maximum iterations reached.');
```

Secant Method Using a WHILE Loop

```

clc;
clear;

% Define the function
f = @(x) x - 2*sin(x);

% Initial guesses
x0 = 1;
x1 = 2;

% Tolerance and maximum iterations
tol = 1e-6;
maxIter = 100;

% Initialize iteration counter and error
i = 0;
error = inf;

% Secant method using WHILE loop
while error > tol && i < maxIter

    % Evaluate the function
    f0 = f(x0);
    f1 = f(x1);
```

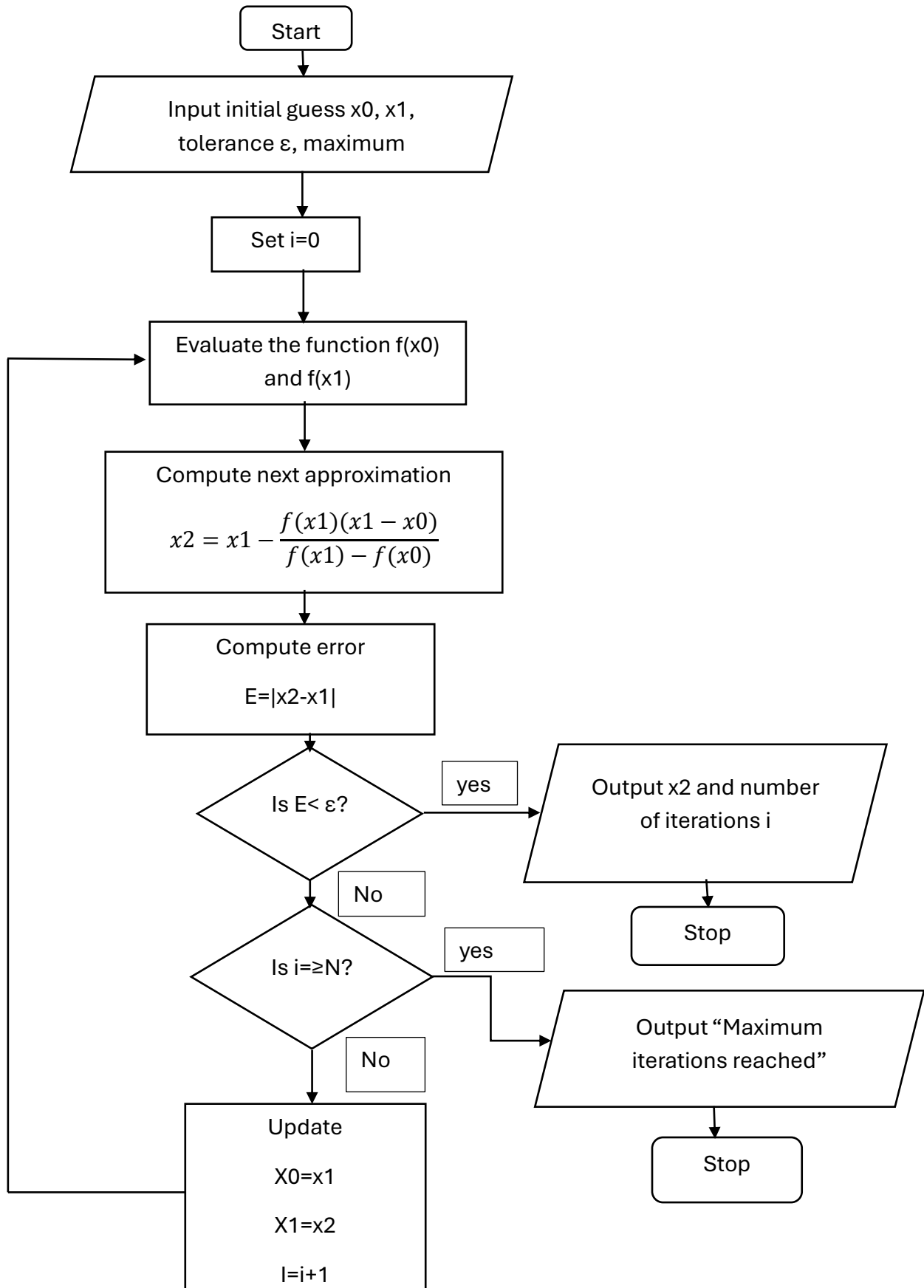
```
% Check for division by zero
if f1 - f0 == 0
    disp ('Division by zero. Method failed. ');
    break;
end

% Calculate the next approximation
x2 = x1 - f1*(x1 - x0)/(f1 - f0);

% Calculate the error
error = abs(x2 - x1);

% Update the guesses
x0 = x1;
x1 = x2;
```

Flow chart for solving the equation using the secant method



Pseudo code for the secant method flowchart

START

Define $f(x) = x - 2\sin(x)$

Input x_0 , x_1 , tolerance TOL

Input maximum iterations N

Set $i = 1$

WHILE $i \leq N$

 Calculate $f_0 = f(x_0)$

 Calculate $f_1 = f(x_1)$

 IF $f_1 - f_0 = 0$ THEN

 Display "Division by zero"

 STOP

 END IF

 Calculate $x_2 = x_1 - f_1(x_1 - x_0)/(f_1 - f_0)$

 IF $\text{abs}(x_2 - x_1) < \text{TOL}$ THEN

 Display x_2

 STOP

 END IF

 Set $x_0 = x_1$

 Set $x_1 = x_2$

 Set $i = i + 1$

END WHILE

Display "Maximum iterations reached"

STOP

Runge- Kutta method

Use the forth order Runge – Kutta algorithm with step size 0.1 to find a numerical solution of the initial value problem;

$$y'' - 2xy' + 8y = 0 \text{ with } y(0) = 12 \text{ and } y'(0) = 0 \text{ in the interval } 0 \leq x \leq 1$$

n	xn	Yn	k1n	xn + (1/2)h	yn+(1/2)k1n	k2n	yn+(1/2)k2n	k3n	xn+h	yn+k3n	k4n	dn	yn+1
0	0	1	-0.4	0.1	0.8	-0.2609	0.86955202	-0.28872	0.2	0.711283	-0.17158	-0.27847	0.721532
1	0.2	0.721532	-0.17568	0.3	0.63368955	-0.09681	0.67312641	-0.11259	0.4	0.608946	-0.05717	-0.10861	0.612924
2	0.4	0.612924	-0.05876	0.5	0.58354306	-0.03392	0.59596482	-0.03889	0.6	0.574037	-0.03485	-0.03987	0.573054
3	0.6	0.573054	-0.03445	0.7	0.55582826	-0.04969	0.54820966	-0.04664	0.8	0.526412	-0.07547	-0.05043	0.522623
4	0.8	0.522623	-0.07396	0.9	0.48564483	-0.10878	0.46823217	-0.10182	1	0.420806	-0.1401	-0.10588	0.416748
5	1	0.416748	-0.13848	1.1	0.34751014	-0.17055	0.33147108	-0.16414	1.2	0.25261	-0.18955	-0.16623	0.250514
6	1.2	0.250514	-0.18871	1.3	0.1561588	-0.20002	0.15050518	-0.19776	1.4	0.052758	-0.19542	-0.19661	0.053902
7	1.4	0.053902	-0.19588	1.5	-0.0440363	-0.17789	-0.0350442	-0.18149	1.6	-0.12759	-0.1482	-0.17714	-0.12324
8	1.6	-0.12324	-0.14994	1.7	-0.1982064	-0.10588	-0.1761776	-0.11469	1.8	-0.23793	-0.05938	-0.10841	-0.23165
9	1.8	-0.23165	-0.06189	1.9	-0.2625949	-0.0051	-0.2341976	-0.01646	2	-0.24811	0.043359	-0.01027	-0.24192
10	2	-0.24192	0.040886	2.1	-0.2214798	0.091955	-0.1959455	0.081741	2.2	-0.16018	0.126381	0.085776	-0.15615
11	2.2	-0.15615	0.124767	2.3	-0.093763	0.153193	-0.0795498	0.147508	2.4	-0.00864	0.162189	0.14806	-0.00809
12	2.4	-0.00809	0.161968	2.5	0.07289737	0.158441	0.07113375	0.159146	2.6	0.15106	0.139285	0.156071	0.147985

MATLAB code for solving the differential equation using the Runge kutta

MATLAB Code Using a FOR Loop

```
clc;
clear;
close all;

% Initial conditions
x0 = 0;
y0 = 12;
v0 = 0;

% Interval and step size
xf = 1;
h = 0.1;

% Number of steps
n = round((xf - x0)/h);

% Create arrays
x = zeros(1,n+1);
y = zeros(1,n+1);
v = zeros(1,n+1);

% Store initial values
x(1) = x0;
y(1) = y0;
```

```

v(1) = v0;

% RK4 method using FOR loop
for i = 1:n

    % First-order derivatives
    k1y = v(i);
    k1v = 2*x(i)*v(i) - 8*y(i);

    k2y = v(i) + h*k1v/2;
    k2v = 2*(x(i)+h/2)*(v(i)+h*k1v/2) ...
        - 8*(y(i)+h*k1y/2);

    k3y = v(i) + h*k2v/2;
    k3v = 2*(x(i)+h/2)*(v(i)+h*k2v/2) ...
        - 8*(y(i)+h*k2y/2);

    k4y = v(i) + h*k3v;
    k4v = 2*(x(i)+h)*(v(i)+h*k3v) ...
        - 8*(y(i)+h*k3y);

    % Update y and its first derivative
    y(i+1) = y(i) + h/6*(k1y+2*k2y+2*k3y+k4y);
    v(i+1) = v(i) + h/6*(k1v+2*k2v+2*k3v+k4v);

    % Update x
    x(i+1) = x(i) + h;
end

% Display results
fprintf('      x          y          dy/dx\n');
disp([x' y' v']);

% Plot the solution
plot(x,y,'o-','LineWidth',1.5);
xlabel('x');
ylabel('y');
title('RK4 Solution using FOR Loop');
grid on;

```

MATLAB Code Using a WHILE Loop

```

clc;
clear;
close all;

% Initial conditions
x = 0;
y = 12;
v = 0;

```

```

% Interval and step size
xf = 1;
h = 0.1;

% Initialize arrays
X = x;
Y = y;
V = v;

% Initialize iteration counter
i = 0;

% RK4 method using WHILE loop
while x < xf - h/2

    % First-order derivatives
    k1y = v;
    k1v = 2*x*v - 8*y;

    k2y = v + h*k1v/2;
    k2v = 2*(x+h/2)*(v+h*k1v/2) ...
        - 8*(y+h*k1y/2);

    k3y = v + h*k2v/2;
    k3v = 2*(x+h/2)*(v+h*k2v/2) ...
        - 8*(y+h*k2y/2);

    k4y = v + h*k3v;
    k4v = 2*(x+h)*(v+h*k3v) ...
        - 8*(y+h*k3y);

    % Update y and its first derivative
    y = y + h/6*(k1y+2*k2y+2*k3y+k4y);
    v = v + h/6*(k1v+2*k2v+2*k3v+k4v);

    % Update x
    x = x + h;

    % Store results
    i = i + 1;
    X(i+1) = x;
    Y(i+1) = y;
    V(i+1) = v;
end

% Display results
fprintf('      x          y          dy/dx\n');
disp([X' Y' V']);

% Plot the solution
plot(X,Y,'o-','LineWidth',1.5);
xlabel('x');

```

```
ylabel('y');  
title('RK4 Solution using WHILE Loop');  
grid on;
```

Pseudo code for Runge Kutta method flow chart

Let $y_1 = y$ and $y_2 = y'$

Then: $y_1' = y_2$

$y_2' = 2x*y_1 - 8*y_1$

START

Define $f_1(x, y_1, y_2) = y_2$

Define $f_2(x, y_1, y_2) = 2x*y_1 - 8*y_1$

Set $x = 0$

Set $y_1 = 12$

Set $y_2 = 0$

Input step size h

Set $x_f = 1$

WHILE $x < x_f$

Calculate $k_1 = h*f_1(x, y_1, y_2)$

Calculate $l_1 = h*f_2(x, y_1, y_2)$

Calculate $k_2 = h*f_1(x+h/2, y_1+k_1/2, y_2+l_1/2)$

Calculate $l_2 = h*f_2(x+h/2, y_1+k_1/2, y_2+l_1/2)$

Calculate $k_3 = h*f_1(x+h/2, y_1+k_2/2, y_2+l_2/2)$

Calculate $l_3 = h*f_2(x+h/2, y_1+k_2/2, y_2+l_2/2)$

Calculate $k_4 = h*f_1(x+h, y_1+k_3, y_2+l_3)$

Calculate $l_4 = h*f_2(x+h, y_1+k_3, y_2+l_3)$

Update $y_1 = y_1 + (k_1+2*k_2+2*k_3+k_4)/6$

Update $y_2 = y_2 + (l_1+2*l_2+2*l_3+l_4)/6$

Update $x = x + h$

Display x, y_1, y_2

END WHILE

STOP

flow chart for solving a differential equation using Runge kutta method

